CAP theorem

The **CAP theorem** states that a distributed data store can simultaneously guarantee at most two of the following three properties:

- **C**onsistency (all nodes see the same data at the same time),
- **A**vailability (every request receives a response), and
- **P**artition Tolerance (the system continues to operate despite network failures).

The Three Core Properties

- **Consistency (C):** Every read request receives the most recent write or an error. If a user updates their profile on Node A, reading that data from Node B will yield the exact same update.
- **Availability (A):** Every request receives a non-error response, even if it cannot guarantee that the data is the most recent version. The system is always up and responding.
- **Partition Tolerance (P):** The system continues to operate despite arbitrary message loss or failure of part of the network.

The Trade-offs (CP vs. AP): the big war

Since **Partition Tolerance (P)** must be assumed in any real-world distributed network, developers must choose one out of the other two: **Consistency** and **Availability.** Hence we have two broader categories of systems around this:

CP (Consistency + Partition Tolerance)

The system prioritizes consistent data over continuous access. If nodes lose connection or cannot sync, the system will block or fail requests rather than serving outdated or conflicting data. [1]

- **Best for:** Applications where data accuracy is paramount, such as financial/banking transactions or seat-booking systems.
- **Examples:** MongoDB, Apache HBase

AP (Availability + Partition Tolerance)

The system prioritizes continuous uptime and responsiveness over exact accuracy. If nodes are partitioned, the system will still accept reads and writes, meaning different nodes might serve slightly different or "stale" data until the network heals.

- **Best for:** Applications where high traffic and uptime are more critical than momentary perfect consistency, such as social media feeds or e-commerce shopping carts.
- **Examples:** Apache Cassandra, Apache CouchDB

Mark your feedback at the Following Contact Points

- **LinkedIn:** https://www.linkedin.com/in/rishirajopenminds
- **GitHub:** https://github.com/rishiraj88

Credits and Gratitude

- Thank all who have mentored, taught and guided. Also, always appreciate who have supported my work.